Lenguajes de Programación

Primera Prueba Presencial

INTRODUCCIÓN + PROGRAMACIÓN FUNCIONAL

Fecha: 21 de octubre de 2013

Valor de la prueba: 20%

Advertencia Importante: Observa bien la sintaxis en los ejemplos. No se considerarán soluciones que utilicen funciones auxiliares y/o no mantengan la sintaxis de la llamada y de los resultados.

EJERCICIO 1 (2 puntos)

Sin utilizar funciones auxiliares, definir una función en lisp llamada ordena que acepte tres valores como parámetros y devuelva una lista con los valores ordenados de menor a mayor.

```
> (ordena -2 4 1)
(-2 1 4)
> (ordena -2 -3 1)
(-3 -2 1)
> (ordena 14 1 9)
(1 9 14)
```

EJERCICIO 2 (2,5 puntos)

Sin utilizar funciones auxiliares, escribe una función en LISP llamada **maxmin** que acepte como único parámetro una lista de valores (positivos o negativos), y devuelva una lista de dos elementos con el valor mayor y menor de los valores (en este orden). La función deberá devolver NIL si la lista proporcionada como parámetro estuviera vacía.

```
> (maxmin '())
NIL
> (maxmin '(8))
(8 8)
> (maxmin '(8 -16))
(8 -16)
> (maxmin '(8 3 3 2 1 9 3 7))
(9 1)
> (maxmin '(8 -1 16 -2 -3))
(16 -3)
```

EJERCICIO 3 (2,5 puntos)

Sin utilizar funciones auxiliares, escribe una función en lisp llamada **vecescero** que acepte una función y una lista de valores como parámetros, y devuelva el número de veces que la aplicación de la función sobre un elemento de la lista evalúa a cero.

Nota. En este caso, los valores se proporcionan en forma de lista. Por lo tanto, la función tiene dos parámetros.

```
> (vecescero #'1+ NIL)
0
> (vecescero #'1+ '(0))    ; Cuenta el número de -1s
0
> (vecescero #'1+ '(-1))     ; Cuenta el número de -1s
1
> (vecescero #'1+ '(-1 2 4 -1))  ; Cuenta el número de -1s
2
> (vecescero #'1- '(1 2 1 4 -1)) ; Cuenta el número de 1s
2
> (vecescero (lambda(x)(floor (/ x 6))) '(1 21 1 4 8 12))
; cuenta valores positivos menores que 6
3
```

EJERCICIO 4 (3 puntos)

Sin utilizar funciones auxiliares, define una función en LISP llamada **calculaf** que acepte como parámetros una función unaria seguida de una serie de valores, y devuelva como resultado una lista cuyos elementos sean el resultado de aplicar la función a cada uno de los valores.

Nota. El resultado es una lista, pero los valores se proporcionan como argumentos. Por lo tanto, la función tendrá un número variable de parámetros.

```
> (calculaf (lambda(x)(+ 1 x)) 1 2 3 4)
(2 3 4 5)
> (calculaf (lambda(x)(+ 1 x)) 1)
(2)
> (calculaf (lambda(x)(+ 1 x)))
NIL
> (calculaf #'1- 1 2 3 4)
(0 1 2 3)
```